GenAI Foundations — Complete Study Guide

Topic: How LLMs Work — Tokens, Context Windows, Pretraining vs Instruction Tuning
Estimated Study Time: 6–8 hours
Output Goal: Short notes + Tokenization demo understanding

BEST VIDEO RESOURCES (Curated & Ranked)

Tier 1 — Must Watch (Core Understanding)

#	Video	Creator	Duration	Link
1	**Deep Dive into LLMs like ChatGPT**	Andrej Karpathy	3h 31m	YouTube
2	**But what is a GPT? Visual intro to Transformers**	3Blue1Brown	27 min	YouTube
3	**Attention in Transformers, step-by-step**	3Blue1Brown	26 min	YouTube
4	**Let's Build the GPT Tokenizer**	Andrej Karpathy	2h 13m	YouTube

Tier 2 — Deep Dive (For builders)

#	Video	Creator	Duration	Link
5	**Let's build GPT from scratch**	Andrej Karpathy	1h 57m	YouTube
6	**Let's reproduce GPT-2 (124M)**	Andrej Karpathy	4h	YouTube
7	**Intro to Large Language Models**	Andrej Karpathy	1h	YouTube

Tier 3 — Supplementary

#	Resource	Type	Link
8	LLM Visualization (Interactive 3D)	Web Tool	bbycroft.net/llm
9	HuggingFace NLP Course	Free Course	huggingface.co/learn
10	mlabonne/llm-course	GitHub Repo	github.com/mlabonne/llm-course

---

STUDY MATERIAL — DETAILED EXPLANATIONS

Module 1: What is a Large Language Model (LLM)?

An LLM is a neural network trained on massive amounts of text data that learns to predict the next word (or token) in a sequence. At its core, an LLM is doing one thing: given all the text so far, what is the most probable next piece of text?

Key Insight: Despite this simple objective, this "next token prediction" task forces the model to develop rich internal representations of grammar, facts, reasoning, and even world knowledge. It's like how learning to predict chess moves forces you to understand strategy — the model must understand language deeply to predict it well.

How an LLM generates text:

1. You give the model a prompt like: "*The capital of France is*"
2. The model outputs a **probability distribution** over all possible next tokens
3. It might say: Paris (92%), located (3%), known (2%)...
4. A token is **sampled** from this distribution (often Paris)
5. That token is appended to the input, and the process **repeats**

This is the **predict → sample → append → repeat** loop. Every chatbot, code assistant, and AI writer is fundamentally running this loop.

Scale matters:

Model	Parameters	Training Data
GPT-2 (2019)	1.5 Billion	~40 GB text
GPT-3 (2020)	175 Billion	~570 GB text
Llama 3.1 (2024)	405 Billion	~15 Trillion tokens
GPT-4 (2023)	Estimated 1.8T (MoE)	Undisclosed

More parameters and more data generally lead to more capable models, though architecture and training quality matter enormously too.

Module 2: Tokens & Tokenization

What is a Token?

A token is the fundamental unit that LLMs operate on. LLMs don't see characters or words — they see **tokens**. A token is a chunk of text (could be a word, part of a word, a single character, or even a space).

Why not just use characters?

If we used individual characters, the sequence "Hello World" would be 11 tokens. That's very long for very little information. The model would need enormous context windows to process even short paragraphs, and each character carries very little meaning on its own.

Why not just use whole words?

English has hundreds of thousands of words. New words (like "ChatGPT") appear constantly. Misspellings, other languages, code, and special characters would all need separate handling. The vocabulary table would be enormous.

The Solution: Byte Pair Encoding (BPE)

BPE is the tokenization algorithm used by most modern LLMs (GPT, Llama, Claude, etc.). It finds a sweet spot between characters and words.

How BPE works step-by-step:

1. **Start with bytes:** Begin with the 256 possible byte values (0–255) as your base vocabulary
2. **Count pairs:** Look at your training text and count every adjacent pair of tokens
3. **Merge the most frequent pair:** The most common pair becomes a new single token
4. **Repeat:** Go back to step 2 with the updated vocabulary
5. **Stop** when you reach your desired vocabulary size (e.g., 50,000 or 100,000 tokens)

Concrete Example:

```
Training text: "low lower lowest low"

Round 1: Most frequent pair is ('l', 'o') → merge into 'lo'
Round 2: Most frequent pair is ('lo', 'w') → merge into 'low'
Round 3: Most frequent pair is ('e', 'r') → merge into 'er'
... and so on
```

Tokenization in Practice

Example using GPT-style tokenization:

```
Input:  "Hello, how are you?"
Tokens: ["Hello", ",", " how", " are", " you", "?"]
IDs:    [15496, 11, 703, 527, 499, 30]
```

Notice that `" how"` includes the leading space — that space is part of the token. This is a common source of confusion.

Tricky cases:

```
"ChatGPT"     → ["Chat", "G", "PT"]          (3 tokens)
"indivisible" → ["ind", "iv", "isible"]      (3 tokens)
"日本語"       → multiple tokens per character   (expensive!)
```

Why Tokenization Matters

Tokenization has real consequences for model behavior:

- **Cost:** API pricing is per-token. More tokens = more expensive.
- **Context limits:** Models have a fixed context window measured in tokens, not words. Roughly, 1 token is about 0.75 English words.
- **Non-English languages** often require more tokens per word, making them more expensive and eating up more context.
- **Spelling struggles:** Since the model sees tokens like "ind" + "iv" + "isible", it doesn't naturally "see" individual characters. This is why LLMs struggle with tasks like "How many r's are in strawberry?" — they're processing token chunks, not individual letters.
- **Code:** Programming languages tokenize differently than English. Python might be more efficient than, say, Ruby depending on the tokenizer's training data.

Try It Yourself — Tokenization Demo Tools

- **OpenAI Tokenizer:** platform.openai.com/tokenizer — Paste text and see how GPT models tokenize it
- **Tiktokenizer:** tiktokenizer.vercel.app — Interactive BPE tokenizer visualization
- **Karpathy's minBPE:** github.com/karpathy/minbpe — Build your own tokenizer in Python

---

Module 3: Context Windows

What is a Context Window?

The context window is the **maximum number of tokens** a model can process at once — it's the model's "working memory." Everything the model reads (your prompt, the system message, previous conversation turns) and everything it writes must fit within this window.

How it works internally

Transformers use a mechanism called **self-attention** where every token can "look at" every other token in the context. This is what allows the model to understand relationships between distant words. But this comes at a cost: the computation scales **quadratically** with context length (if you double the context, the attention computation roughly quadruples).

Context Window Sizes (as of 2025)

Model	Context Window
GPT-2 (2019)	1,024 tokens
GPT-3 (2020)	2,048 tokens
GPT-4 (2023)	8,192 / 32,768 / 128,000 tokens
Claude 3.5 (2024)	200,000 tokens
Gemini 1.5 Pro (2024)	1,000,000+ tokens
Llama 3.1 (2024)	128,000 tokens

Rough conversion: 100,000 tokens is about 75,000 words, or roughly a 300-page book.

Why Context Windows Matter

- **Conversation memory:** In a chatbot, the entire conversation history must fit in the context window. When it overflows, older messages get dropped — the model literally "forgets."
- **Document analysis:** Can the model read your entire PDF at once, or does it need to process it in chunks?
- **Code understanding:** Can it see your entire codebase or just snippets?

What happens at the boundary?

When the context window fills up, models (or the applications built on them) use strategies like:

- **Truncation:** Simply cut off the oldest content
- **Sliding window:** Keep the most recent N tokens
- **Summarization:** Compress old context into a shorter summary
- **Retrieval-Augmented Generation (RAG):** Store information externally and retrieve relevant chunks as needed

The "Lost in the Middle" Problem

Research has shown that LLMs pay the most attention to information at the **beginning** and **end** of the context

window, with reduced attention to content in the middle. This means that where you place important information in your prompt actually matters for performance.

Module 4: The Transformer Architecture

The Transformer is the neural network architecture behind all modern LLMs. It was introduced in the 2017 paper "Attention Is All You Need" by Google researchers.

High-Level Flow

```
Input Text
   |
[Tokenization] — break text into tokens
   |
[Embedding] — convert each token into a vector (list of numbers)
   |
[Positional Encoding] — add information about token positions
   |
+-------------------------------------+
|  Repeated N times (layers):         |
|                                     |
|  [Self-Attention Block]             |
|    - Tokens "talk" to each other    |
|    - Each token gathers context     |
|                                     |
|  [Feed-Forward Network (MLP)]       |
|    - Each token is processed        |
|      independently through a        |
|      neural network                 |
|                                     |
+-------------------------------------+
   |
[Unembedding] — convert final vectors back to token probabilities
   |
Output: probability distribution over next token
```

Key Components Explained

1. Embeddings

Each token is mapped to a high-dimensional vector (e.g., 12,288 dimensions for GPT-3). These vectors are learned during training and encode semantic meaning. Words with similar meanings end up with similar vectors — "king" and "queen" are close together in this space, as are "dog" and "puppy."

2. Self-Attention ("The Core Innovation")

Self-attention is what makes transformers powerful. It allows every token to "look at" every other token and decide which ones are relevant for its current meaning.

Consider the word "bank" in:

- "I sat by the river **bank**" — attention flows to "river"
- "I deposited money at the **bank**" — attention flows to "money" and "deposited"

The attention mechanism uses three learned projections for each token:

- **Query (Q):** "What am I looking for?"
- **Key (K):** "What do I contain?"
- **Value (V):** "What information do I provide?"

The attention score between two tokens is the dot product of one token's Query with another token's Key. High scores mean those tokens are relevant to each other. The Values are then mixed according to these scores.

3. Multi-Head Attention

Instead of one attention mechanism, transformers run many in parallel (e.g., 96 "heads" in GPT-3). Each head can learn to attend to different types of relationships — one might track grammatical structure, another might track co-reference ("it" refers to what?), another might track semantic similarity.

4. Feed-Forward Network (MLP)

After attention, each token passes independently through a neural network. Think of this as "processing" the information gathered from attention. Research suggests this is where much of the model's factual knowledge is stored.

5. Layer Stacking

All of this — attention + MLP — is repeated many times (96 layers in GPT-3). Early layers tend to capture simple patterns (syntax, grammar). Middle layers capture more complex patterns (semantics, entity relationships). Later layers capture the most abstract patterns (reasoning, task-specific behavior).

Module 5: Pretraining vs Instruction Tuning

This is one of the most important distinctions in modern AI. A model goes through multiple training stages, each giving it different capabilities.

Stage 1: Pretraining

What: The model reads enormous amounts of internet text and learns to predict the next token.

Data: Web crawls (Common Crawl), Wikipedia, books, code repositories, scientific papers — trillions of tokens.

Duration: Weeks to months on thousands of GPUs. Training Llama 3.1 405B reportedly cost over $100M in compute.

What it produces: A **base model** (also called a "foundation model" or "pretrained model"). This model is like a brilliant student who has read the entire internet but has no idea what a conversation is. If you prompt it with "What is the capital of France?", it might complete it as a trivia quiz question and add "What is the capital of Germany?" rather than answering.

Example base model behavior:

```
Prompt:  "What is the capital of France?"
Output:  "What is the capital of Germany? What is the capital
          of Spain? These are common geography quiz questions..."
```

The base model is a powerful **text completer**, not yet a helpful **assistant**.

Stage 2: Supervised Fine-Tuning (SFT) / Instruction Tuning

What: The base model is further trained on curated datasets of high-quality (prompt, response) pairs — often written by human contractors.

Data: Tens of thousands to millions of examples like:

```
User:    "Explain quantum computing to a 10-year-old."
Assistant: "Imagine a regular computer is like a light switch —
           it can be ON or OFF. A quantum computer is like a
           magical light switch that can be ON, OFF, or both
           at the same time!..."
```

What it produces: An **instruction-tuned model** (or "chat model") that understands conversation format, follows instructions, is helpful, and knows when to refuse harmful requests.

Key transformation: The model shifts from "complete this text" to "answer this question helpfully."

Stage 3: Reinforcement Learning from Human Feedback (RLHF)

What: Humans rate different model outputs. A reward model is trained on these preferences, and then the LLM is further optimized to produce outputs that score highly.

Process:

1. Model generates multiple responses to the same prompt
2. Human raters rank them from best to worst
3. A "reward model" learns these preferences
4. The LLM is trained to maximize the reward model's score (using PPO, DPO, or similar RL algorithms)

What it produces: A model that is not just instruction-following but is **aligned** — it's helpful, harmless, and honest. It handles edge cases better, produces more natural responses, and avoids problematic outputs.

Comparison Table

Aspect	Pretraining	Instruction Tuning (SFT)	RLHF
Goal	Learn language & knowledge	Learn to follow instructions	Align with human preferences
Data	Trillions of tokens (internet)	Thousands of curated examples	Human preference rankings
Compute	Enormous (months, 1000s of GPUs)	Moderate (days, 10s of GPUs)	Moderate
Result	Base model (text completer)	Chat model (assistant)	Aligned model (safe assistant)
Analogy	Reading every book in the library	Learning to be a tutor	Learning what makes a *good* tutor

Stage 4 (Emerging): Reinforcement Learning for Reasoning

Models like DeepSeek-R1 and OpenAI's o1/o3 add a further training stage specifically focused on reasoning. The model learns to "think step by step" in an extended chain-of-thought before answering. This dramatically improves performance on math, logic, and coding tasks.

SUGGESTED 6-8 HOUR STUDY PLAN

Time Block	Activity	Duration
Hour 1	Watch: Karpathy "Intro to LLMs" (1h)	1h
Hour 2	Watch: 3Blue1Brown "Visual intro to Transformers" (27min) + "Attention" (26min)	1h
Hour 3	Read: Module 1-3 of this guide (LLMs, Tokens, Context Windows)	1h
Hour 4	Hands-on: Play with OpenAI Tokenizer & Tiktokenizer. Tokenize different languages, code, and edge cases	1h
Hour 5-6	Watch: Karpathy "Deep Dive into LLMs" — Sections on pretraining, SFT, and RLHF (select chapters)	1.5-2h
Hour 7	Read: Module 4-5 of this guide (Transformers, Pretraining vs Tuning)	1h
Hour 8	Review: Write your own short notes summarizing each module in your own words	1h

---

QUICK REFERENCE — SHORT NOTES

LLMs in one sentence: LLMs are neural networks trained to predict the next token, which forces them to learn grammar, facts, and reasoning from massive text datasets.

Tokens in one sentence: Tokens are sub-word chunks created by BPE that serve as the atomic units of input/output for LLMs (roughly 1 token per 0.75 English words).

Context window in one sentence: The context window is the fixed-size "memory" (measured in tokens) that limits how much text a model can read and generate at once.

Transformer in one sentence: A Transformer uses self-attention (tokens look at all other tokens) and feed-forward layers, stacked many times, to convert input tokens into next-token predictions.

Pretraining in one sentence: Pretraining reads trillions of tokens from the internet to create a base model that is knowledgeable but doesn't yet know how to be a helpful assistant.

Instruction tuning in one sentence: Instruction tuning (SFT + RLHF) trains the base model on curated conversation examples and human preference data to create a helpful, safe, aligned assistant.

ADDITIONAL READING & RESOURCES

Resource	Type	Link
"Attention Is All You Need" (original paper)	Paper	arxiv.org/abs/1706.03762
The Illustrated Transformer (Jay Alammar)	Blog	jalammar.github.io
Brendan Bycroft LLM Visualization	Interactive	bbycroft.net/llm
Karpathy's minBPE repo	Code	github.com/karpathy/minbpe
mlabonne/llm-course	Roadmap	github.com/mlabonne/llm-course
HuggingFace NLP Course	Course	huggingface.co/learn/nlp-course
Sebastian Raschka — Build LLM from Scratch	Book/Videos	magazine.sebastianraschka.com

---

Last updated: February 2026.